

Axona Applications

What runs on Axona today, what gets better if it migrates, and what we are building next.

Version 0.1 · 2026-06-01 · David A. Smith · davidasmith@gmail.com

How to read this document

This document has two parts.

Part I — Existing Applications: A Deep Dive examines the dozen applications that depend on a DHT today, and what changes for each if its discovery/propagation layer moves to Axona. It is adapted from the whitepaper chapter of the same name and refreshed to the **current state of the protocol**: Axona is no longer a simulator result, it is a live, browser-native network (@axona/protocol **v2.10.0**) with an authenticated handshake, signed and freshness-bounded messages, and a full pub/sub lifecycle. Where that shipped reality strengthens (or qualifies) the original projection, the text says so.

Part II — Future Applications: Axona Powered covers the applications we are building *on* Axona rather than migrating *to* it — *civildefense.io* (the first end-user app), **SYZL** (the adaptive social feed), and a deliberately open pipeline for what comes next.

> **On the numbers.** Latency figures in Part I are back-of-envelope > projections from the 25,000-node benchmark in the whitepaper > (§ Performance), applied to each application's typical workload. They are > projections, not field measurements of the migrated application, and are > marked as such. What is *not* a projection is the protocol surface itself — > the API verbs, the security properties, the lifecycle semantics — which are > live and testable today.

Part I — Existing Applications: A Deep Dive

For each application: what it does, the DHT layer it uses now, that layer's performance profile, where it falls short of user expectations, and what changes if it migrates to Axona. *Streaming-class applications — those that become possible only with Axona's latency budget but do not exist today — are the subject of Part II, not this part.*

What "migrate to Axona" means in 2026

When the original chapter was written, "N-DHT / NH-1" was a simulator. The claims below now rest on a deployed substrate, which changes their character from *hypothesis* to *engineering estimate*:

- **Browser-native and live.** Axona was designed for the WebRTC environment from the start. The reference network runs at <https://axona.net> via the axona-peer browser node and the axona-bridge signaling broker. The bridge carries WebRTC offer/answer payloads only and then drops out — it sees no application traffic, and any operator can run one (a federated bridge mesh is on the roadmap).

- **Authenticated, not anonymous-by-accident.** Every peer link is established

through the axona/4 authenticated handshake, with the mesh channel-binding value bound to the DTLS certificate fingerprints — so a relaying bridge cannot transparently MITM "direct" peer traffic.

- **Signed and fresh.** Every published message is an Ed25519-signed envelope

(format v2) over a domain-tagged core carrying a per-publisher monotonic sequence number and timestamp, with a freshness window enforced at ingress. Replay-to-fresh-subscribers is closed.

- **A real lifecycle, not just fan-out.** Beyond pub/sub, the kernel now

exposes unsubs, pull (by id or latest), kill (creator-only delete with tombstones), unpub (owner-only topic destroy), bounded per-topic queues with deterministic eviction, and a hold-time TTL (24 h default, 48 h ceiling, sliding on pull). Membership-gated topics are supported via shared-encryption group keys at the application layer.

With that substrate in place, the per-application analysis follows.

BitTorrent (Mainline DHT)

What it does. Mainline DHT is BitTorrent's trackerless peer-discovery layer: a torrent's `info_hash` maps into the keyspace, swarm peers register under that key, and new peers query the DHT to find the swarm.

Current implementation. Kademlia, 160-bit IDs, $K=8$ buckets, $\alpha=3$ parallel queries. The largest production DHT in the world by node count (estimated 10–20 M nodes at peak).

Performance & shortfall. Cold-start `get_peers` latency is typically 5–30 s, dominated by the iterative α -parallel walk through globally-scattered random-ID peers. Modern clients hide this behind a centralized tracker for the first discovery round; the DHT is the tracker-resistance *backup*, not the primary path — precisely because the cold-start latency is unacceptable for a primary path.

What Axona changes. Mainline lookups translate directly (same operation, same data model). Projected lookup latency drops to **~250–300 ms at 25K nodes** — a 20–100× improvement that collapses the user-visible delay below the threshold where tracker fallback adds value. The qualitative shift: the DHT becomes *the* discovery layer, not a fallback.

IPFS / Filecoin / libp2p

What it does. Content-addressable storage: content is referenced by a CID, and the libp2p Kad-DHT routes lookups to peers that announced they hold it.

Current implementation. libp2p Kad-DHT (Kademlia variant), default $K=20$, $\alpha=3$, tens of thousands of nodes.

Performance & shortfall. CID lookup averages 1–5 s; provider-record retrieval adds 1–3 s; end-to-end first-byte from cold cache is 5–15 s — 25–300× slower than a centralized HTTP CDN (50–200 ms). The IPFS gateway tier exists to paper over this with centralized servers. Separately, libp2p-pubsub (gossipsub) for IPNS updates carries its own mesh-scaling challenges.

What Axona changes. DHT-layer lookups drop to **~250–300 ms** (projection), making end-to-end first-byte **~500 ms–1 s** — a 10–20× improvement that turns CDN-replacement viability from hypothetical into a real question. Axona's axonal pub/sub trees would replace gossipsub for IPNS updates with bounded per-relay fan-out cost. The combination makes IPFS deployable on browser-class infrastructure for both halves, which it currently is not.

Ethereum (discv5)

What it does. discv5 is Ethereum's peer-discovery layer — how a new node finds peers — using a Kademlia variant with verifiable-identity ENRs.

Performance & shortfall. Cold-start peer acquisition takes 30 s to several minutes (Kademlia walk + per-peer ENR verification round-trips). For an on-demand join (fresh validator, new searcher) that is operational friction; mempool gossip via gossipsub adds a latency-sensitive scaling dimension the static Kademlia table cannot adapt to.

What Axona changes. Discovery latency drops 10–50× for the DHT layer (projection). The locality-aware bootstrap gives new nodes regionally-sensible initial peer sets — which matters more for Ethereum than BitTorrent because mempool propagation is latency-sensitive. Axona's axonal trees are a natural fit for "broadcast a transaction to all interested validators in a region," addressing the gossipsub limitation directly. *Axona's authenticated handshake maps cleanly onto discv5's identity requirements — the migration does not trade away ENR-style verifiable identity.*

Bitcoin Block / Transaction Propagation

What it does. Bitcoin propagates blocks/transactions through a P2P mesh: DNS-seed bootstrap, addr-message peer discovery, gossip propagation.

Performance & shortfall. Full-block propagation averages 1–3 s (compact blocks reduce the announce to a few hundred ms plus re-request); mempool tx latency is 50–200 ms. The flooded gossip is **not locality-aware** — a US-east → Asia transaction may route through Europe — and MEV concentrates around peers with privileged low-latency access to miners.

What Axona changes. Locality-aware discovery solves regional clustering; nodes find nearby peers by default and transactions propagate along short-path edges. Compact-block propagation becomes routed pub/sub: every node subscribes to a block-announcement topic, the miner publishes, the axonal tree fans out at **~100 ms at 50K nodes** (projection). The tree is built once and reused rather than re-running inventory exchange per peer per round, and peer-to-miner latency becomes a property of geography rather than peering relationships — the privileged-peer problem partially dissolves.

Tor Onion Services v3

What it does. Self-authenticating, location-hidden endpoints; descriptor lookup is mediated by the HSDir hash ring.

Performance & shortfall. Rendezvous takes 5–15 s end-to-end, dominated by Tor circuit setup; the descriptor lookup itself adds 1–3 s. HSDir nodes are infrastructure-tier (a centralization concession), and descriptor caching is short-lived — popular services see fetch storms.

What Axona changes. The descriptor lookup itself drops to **~300 ms** (projection), though the Tor circuit still dominates total cost. The structural benefit is decentralizing the HSDir tier: Axona's hop-caching deposits descriptors at any node along the lookup path, widening the descriptor-fetch tier beyond designated relays. The directory-authority trust model is unchanged — Axona widens the cache tier, it does not replace Tor's anonymity design.

Storj / Sia (Decentralized Storage)

What it does. Files are sharded, encrypted, and replicated across provider nodes; the DHT mediates shard placement and retrieval.

Performance & shortfall. Storj shard retrieval is 200–800 ms for a 4 KB shard (dominated by the connection to a remote provider, not the lookup); provider discovery at edge is ~1 s. Placement is **region-blind** — a Berlin user may hold shards in São Paulo.

What Axona changes. Locality-aware placement: shards land on providers in the user's S2 cell with high probability, dropping retrieval to the regional budget (**~100–200 ms**, projection). Repair coordination (replacing a shard when a provider drops) becomes an axonal-tree topic with high delivery under churn — and the new lifecycle verbs give it explicit semantics: a provider going offline is a `kill/tombstone` event, not an inference from silence.

Tox / Briar (Peer-to-Peer Messengers)

What it does. Serverless P2P messengers. Tox uses a Kademlia DHT for contact discovery; Briar uses a hybrid local + overlay routing.

Performance & shortfall. Tox contact-availability checks take 2–10 s; NAT makes initial establishment slower. **Group chat is expensive** — every message goes to every member, making large groups infeasible — and "contact came online" requires polling because there is no real pub/sub.

What Axona changes. Contact discovery drops to **~250 ms** (projection), comparable to a centralized messenger's offline-status check. Group chat becomes a pub/sub topic: bounded per-hop fan-out scaling to thousands of members; "came online" becomes an automatic event. Crucially, the membership and privacy story is now real, not aspirational: a private group is a **shared-encryption topic** (group key distributed at the app layer; the protocol carries opaque ciphertext and gates nothing it shouldn't), and `kill/unsub/unpub` give first-class leave/remove/teardown. This is the first P2P messenger architecture that scales to large groups with a centralized service's latency profile *and* a credible membership model.

YaCy (Decentralized Search)

What it does. A P2P search engine — each peer crawls part of the web; a custom DHT mediates shared indexing and query distribution.

Performance & shortfall. Queries take 2–30 s (index lookup + query distribution, both $\sim \log N$ on a high ~ 200 –500 ms per-peer constant). Against Google's ~ 200 ms median this is unusable interactively — the UX gap is why YaCy stayed a research curiosity.

What Axona changes. DHT-layer query distribution drops to **~ 250 ms** (projection); shard-level result caching via hop caching makes an interactive YaCy-like engine plausible. Ranked aggregation across shards is a separate algorithmic problem, and beating Google needs decades of ranking infrastructure — but the *latency floor* stops being the disqualifying factor.

Radicle (Decentralized Git Forge)

What it does. P2P git collaboration; repositories addressed by peer-ID, a DHT layer for repository discovery, custom routing for pull requests.

Performance & shortfall. Repository discovery is 1–3 s; change-set propagation is 5–30 s — making "find a project, browse it" feel slow vs GitHub and "watch this repo" poll-based rather than push-based.

What Axona changes. Repository discovery drops to **~ 300 ms** (projection); change-set propagation becomes a commits-on-this-repo axonal topic with **~ 200 ms** fan-out, making the forge interactive in the GitHub sense, not just browsable. The signed-envelope sequence numbers give watchers a tamper-evident, gap-detectable commit feed for free.

Handshake / ENS (Decentralized DNS)

What it does. Decentralized name resolution; ownership recorded on-chain, a DHT-like layer caches recent lookups.

Performance & shortfall. Hot cached lookups are < 100 ms; cold lookups hit the chain and take seconds to minutes, so cache hit-rate dominates. The cache tier is operated by infrastructure peers (a centralization tradeoff) because random-Kademlia caching does not scale.

What Axona changes. Hop caching makes the cache tier *implicit*: every routed lookup deposits its result at intermediate nodes, popular names accrue shortcuts naturally, and the explicit caching infrastructure becomes unnecessary. Cold-cache latency drops because the routing itself is faster.

WebTorrent / WebRTC P2P

What it does. BitTorrent for browsers over WebRTC data channels, bridged to Mainline DHT for peer discovery (most browsers cannot speak Mainline directly).

Performance & shortfall. Discovery is 5–30 s (Mainline latency + bridge overhead). The **Mainline-DHT bridge is a centralization concession driven entirely by the protocol's inability to run in browsers** — the bridge operators are a single point of failure and surveillance.

What Axona changes. Axona *is* a browser-native DHT — this is the cleanest fit in the catalogue. WebTorrent on Axona eliminates the *discovery* bridge: peer discovery runs in-browser at the regional budget (**100–300 ms**), and the browser becomes a first-class P2P participant. (Axona still uses a lightweight *signaling* bridge for WebRTC NAT traversal, but it carries no application or discovery traffic and drops out once peers are connected — a categorically smaller role than the Mainline-DHT bridge it replaces.) The connection-cap analysis shows this is sustainable on browser-class infrastructure.

Mastodon-Style Federations (No DHT Today)

What it does. A federation of independent ActivityPub servers; users follow accounts on other servers, and the home server fetches/serves federated content.

Current implementation. No DHT. Discovery via WebFinger (centralized per-domain handle resolution); cross-server delivery via direct HTTP push from publisher's home to each subscriber's home.

Performance & shortfall. Home servers (often small VPS) show 200–2000 ms under load; popular-post federation can lag by minutes due to per-subscriber- server fan-out. The **home-server-as-bottleneck** pattern means a viral post on a small instance can take the instance down, because every other instance's followers fetch from the publisher's home.

What Axona changes. This is the application that becomes *architecturally different*. Handle resolution becomes an Axona lookup (independent of home-server load). Status delivery becomes an axonal-tree topic per follower-graph cluster: a viral post fans out via the tree at bounded per-relay cost, never saturating any single home server. Home servers keep durable storage and WebFinger origin authority, but the fan-out load moves to the DHT layer — making small servers stop being single points of failure for their users' reach.

Summary of improvements

Latency numbers are projections from the 25K-node benchmark; production deployments need the friction-realistic simulator extensions to confirm.

Application	Current bottleneck	Axona improvement
BitTorrent (Mainline)	5–30 s cold lookup	~300 ms; primary path possible
IPFS / Filecoin	5–15 s first-byte	~1 s; CDN-replacement viable
Ethereum discv5	Multi-minute join, mempool latency	~1 s join, locality-aware mempool

Bitcoin propagation	Region-blind, MEV-prone	Locality-aware, axonal-tree announce
Tor onion services	DHT minor; HSDir centralization	Wider HSDir tier, faster lookups
Storj / Sia	Region-blind shard placement	Locality-aware shards + repair pub/sub
Tox / Briar messengers	Slow group chat, polling status	Pub/sub group chat, push status, real membership
YaCy	5–30 s queries	~300 ms DHT layer; interactive plausible
Radicle	5–30 s changeset propagation	~200 ms via axonal pub/sub
Handshake / ENS	Cold-cache slow	Implicit caching via hop caching
WebTorrent	Mainline-DHT discovery bridge	Discovery bridge eliminated; browser-native
Mastodon-style federation	Home-server bottleneck	Decentralized fan-out via axonal trees

The pattern. The DHT layer is rarely the *only* bottleneck, but it is consistently a bottleneck, and moving to Axona consistently moves it out of the user-visible critical path. The biggest beneficiaries are the interactive applications (messengers, search, dev forges) where DHT latency sits in the user's path; the least are those where the DHT is already a small fraction of total latency (Tor circuits, Storj's remote-provider connection). The application that becomes *architecturally different* is the federation pattern, because Axona's pub/sub primitive replaces the home-server fan-out those patterns suffer under.

Honest framing. We keep the federation claim modest: the architecture is *enabled*, not necessarily preferable in every dimension. And the latency figures remain projections until measured on the friction-realistic substrate. What changed since the original chapter is the floor under those projections — the protocol is no longer a simulator artifact but a live, authenticated, signed-and-fresh network with a real lifecycle.

Part II — Future Applications: Axona Powered

Part I asked "what gets better if an existing app migrates?" Part II asks the opposite: "what do we build that *only* Axona makes practical?" These are applications designed Axona-native from the first line — they consume the pub/sub lifecycle, the geographic locality, the signed-and-fresh envelopes, and the no-central-operator property as load-bearing features, not conveniences.

civildefense.io — the flagship

What it is. A tap-to-report incident map. A user taps a location to register a concern; reports propagate over anonymous peer-to-peer with a **24-hour expiry**, surfacing on a shared regional map without any central server holding the data.

Why it is Axona-native. Every primitive the app needs is a protocol primitive:

- **Geographic locality is the product.** The S2 cell prefix in every nodeID means a report is relevant to, and routes toward, the people physically near it — at zero coordination cost. An incident map is, structurally, a geographically-keyed pub/sub feed, which is exactly what Axona routes.

- **Expiry is a first-class semantic now, not a side effect.** The original brief leaned on implicit expiry through the replay-cache LRU. With the v2.10.0 lifecycle, the 24-hour window is the **hold-time TTL** (24 h default, 48 h ceiling), an explicit, principled bound — reports age out deterministically rather than whenever the cache happens to evict them.

- **Signed, fresh, and tamper-evident.** Reports are Ed25519-signed envelopes with per-publisher sequence numbers and a freshness window — so a stale or replayed report cannot be re-injected at a fresh subscriber, which matters acutely for an incident feed.

- **Anonymous but authenticated transport.** The axona/4 handshake plus mesh channel-binding means reports travel end-to-end without a central operator and without a relaying bridge being able to read or rewrite them.

What it still needs from us. A membership-gated variant (e.g., a verified responder channel) wants **Model 3 shared-encryption group keys** — the group key is distributed at the application layer and the protocol carries opaque ciphertext. [civildefense.io](#) is one of the two worked references (alongside the open-source reference app) driving that app-layer group-key design.

Status. First end-user application; the primitive fit is why it came together in weeks rather than quarters.

SYZL — the adaptive social feed

What it is. A swipe-based decentralized social feed where every gesture trains your network: **swipe right to** SYZL a post (it reshares to your followers and your connection to the publisher strengthens), **swipe left to** FYZL it (it stops, and the connection weakens). Ranking runs transparently on your own device from your own swipe history — there is no algorithm operator and no server-side ranking. (*Full product brief: SYZL-Brief.md.*)

Why it is Axona-native.

1. **The mechanism mirrors the protocol.** Axona's neuromorphic routing strengthens useful connections and prunes the rest; SYZL does the same one layer up, with user attention as the learning signal. Same algorithm, different layer — the protocol's own story retold for end users.

2. **No central operator, no engagement-maximization incentive.** No ads, no rage-bait amplification, no shadowbanning; ranking runs only on-device.

3. **Verifiable reach without surveillance.** Publishers see reach (publishes + reshare_count) via peer.metrics without learning *who* reshared — the privacy property Axona was built for, surfaced at the app layer.

4. **Connection-sharing as a feature, not a leak.** Because subscriptions are first-class protocol objects, "share my graph" is a native operation; recipients adopt connections selectively, with provenance.

Lifecycle fit. SYZL is almost entirely application-layer over @axona/protocol: pub/sub for posts, reshares, and connection-list cards; pull for referenced bodies; metrics for the reach dashboard;

unsub for dropping a rotated-out connection cleanly. The hill-climb + simulated-annealing exploration that keeps the feed alive is ~200 lines on top of lookup/peers.

SYZL Anywhere. A companion Manifest-V3 browser extension that lets a user SYZL any page, selection, or image in one click, running the user's Axona peer in a background service worker — which doubles as keeping the network warm while the user browses. ~6 weeks, ~70% code-shared with the PWA composer.

The pipeline — others as we invent them

This section is deliberately open. Part I is, in effect, a menu: several of those "what Axona changes" analyses describe applications that are more interesting built *fresh* on Axona than migrated. Natural near-term candidates, none yet committed:

- **A browser-native swarm transport.** WebTorrent's discovery bridge eliminated — large-file or live-media distribution among browser peers with no Mainline-DHT intermediary.
- **A large-group P2P messenger.** Tox/Briar's group-chat ceiling lifted by pub/sub fan-out plus Model 3 shared-encryption membership.
- **A federated-feed fan-out layer.** The Mastodon-style "viral post on a small instance" failure removed by moving fan-out to axonal trees while home servers keep storage and identity.

As each of these graduates from idea to product, it gets its own brief in this folder and a row in Part II.

What the Axona-powered applications share

Across civildefense.io, SYZL, and the pipeline, the same four properties keep recurring as the reason the app is *possible*, not merely *cheaper*:

1. **Geographic locality for free** — the S2 prefix puts relevance and routing in the same place at zero coordination cost.
2. **Pub/sub with a real lifecycle** — pub/sub/unsub/pull/kill/unpub, bounded queues, and hold-time TTL turn "fan a message out" into a managed, expiring, revocable feed.
3. **Signed, fresh, end-to-end-secure messages** — authenticity, freshness, and MITM-resistance are protocol defaults, so the application never has to bolt them on.
4. **No central operator** — ranking, membership, and reach live on-device and in the mesh, which removes the surveillance-and-incentive failure mode common to their centralized counterparts.

These are the same properties Part I shows existing applications *reaching toward* through bridges, gateways, and caching tiers. The forward-looking bet of Part II is that designing for them from the first line produces applications the migration path cannot — because the protocol's defaults are the product's features.